4 React core principles and patterns

This chapter covers

- Declarative UI: Component architecture, JSX, Virtual DOM, Fiber, and React Compiler
- One-way data flow and state management
- React Hooks, the `use` API, and custom hooks
- Component lifecycle, controlled components, Suspense, concurrency, and error handling
- React developer tools, profiler API, and debugging techniques

Jane's experience highlights an important lesson for React developers preparing for technical interviews and building efficient UIs. In a challenging interview, she struggled with questions on React's Virtual DOM, Fiber Architecture, and reconciliation—concepts she had not reviewed recently despite her experience. When asked to implement a tooltip, her solution worked but showed visual flickering, exposing gaps in her practical skills. Determined to improve, Jane revisited React fundamentals like the component lifecycle and learned that switching from `useEffect` to `useLayoutEffect` resolved the flickering. This deeper understanding not only enhanced her implementation's performance but also boosted her confidence in later interviews. Her journey underscores the value of mastering React's internals to optimize app performance, debug effectively, and excel in both interviews and real-world development.

4.1 Declarative UI: The React way

React introduces a declarative approach to UI development, where developers specify what the UI should look like for a given state, and React efficiently manages DOM updates. This simplifies building maintainable, performant interfaces. In interviews, candidates are often asked how React's component model, JSX, and Virtual DOM support this paradigm, and advanced questions may cover Fiber architecture, concurrent rendering, and